

2.

General

Diagramming

Guidelines

The guidelines presented in this chapter are applicable to all types of diagrams, UML or otherwise. The terms “symbols,” “lines,” and “labels” are used throughout:

- Symbols represent diagram elements such as class boxes, object boxes, use cases, and actors.
- Lines represent diagram elements such as associations, dependencies, and transitions between states.
- Labels represent diagram elements such as class names, association roles, and constraints.

2.1 Readability Guidelines

1. Avoid Crossing Lines

When two lines cross on a diagram, such as two associations on a UML class diagram, the potential for misreading a diagram exists.

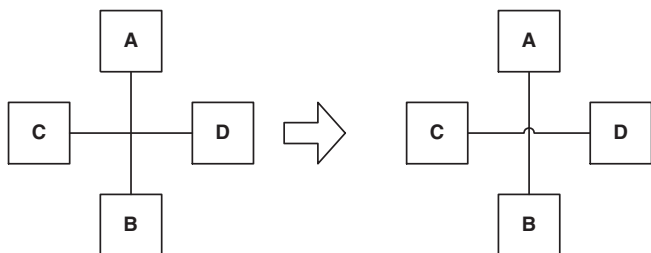


Figure 1. Depiction of crossing lines.

2. Depict Crossing Lines as a Jump

You can't always avoid crossing lines; for example, you cannot fully connect five symbols (try it and see). When you need to have two lines cross, one of them should "hop" over the other as in Figure 1.

3. Avoid Diagonal or Curved Lines

Straight lines, drawn either vertically or horizontally, are easier for your eyes to follow than diagonal or curved lines. A good approach is to place symbols on diagrams as if they are centered on the grid point of a graph, a built-in feature of many computer-aided system-engineering (CASE) tools. This makes it easier to connect your symbols by only using horizontal and vertical lines. Note how three lines are improved in Figure 2 when this approach is taken. Also note how the line between *A* and *C* has been depicted in "step fashion" as a line with vertical and horizontal segments.

4. Apply Consistently Sized Symbols

The larger a symbol appears, the more important it seems to be. In the first version of the diagram in Figure 2, the *A* symbol is larger than the others, drawing attention to it. If that isn't the effect that you want, then strive to make your symbols of uniform size. Because the size of some symbols is

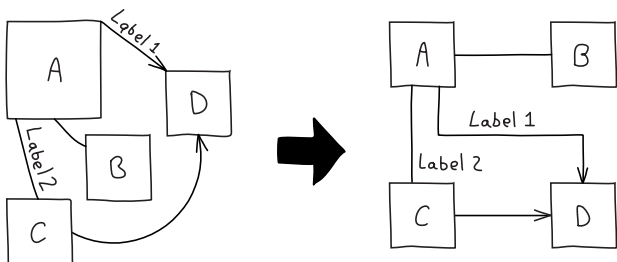


Figure 2. Improving the attractiveness of a diagram.

determined by their contents—for example, a class will vary in size based on its attributes and operations—this rule is not universally applicable. Ideally you should only deviate if you want to accentuate an aspect of your diagram (Koning, Dormann, and Van Vliet 2002).

5. *Attach Lines to the Middle of Bubbles*

As you can see in Figure 2, the *Label 1* line between *A* and *D* is much more readable in the updated version of the diagram.

6. *Align Labels Horizontally*

In Figure 2 the two labels are easier to read in the second version of the diagram. Notice how *Label 2* is horizontal even though the line it is associated with is vertical.

7. *Arrange Symbols Symmetrically*

Figure 3 presents a UML activity diagram (Chapter 10) depicting a high-level approach to enterprise modeling. Organizing the symbols and lines in a symmetrical manner makes the diagram easier to understand. A clear pattern will make a diagram easier to read.

8. *Don't Indicate “Exploding” Bubbles*

The rake symbol in the upper right corner of each activity in Figure 3 is the UML way to indicate that they “explode” to

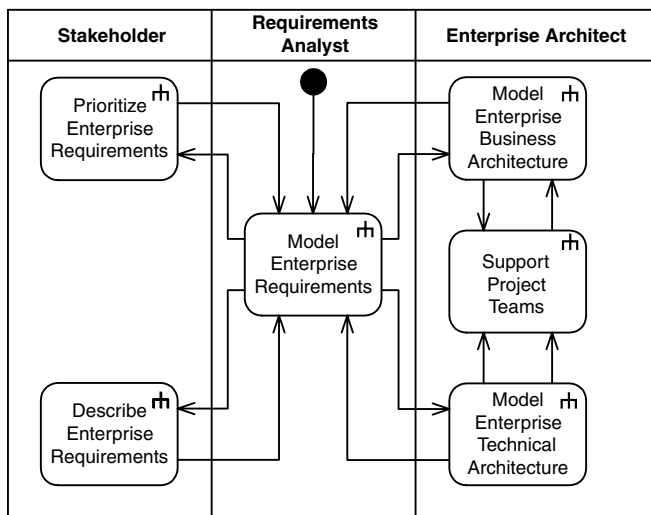


Figure 3. UML activity diagram for a software process.

another diagram showing a greater level of detail. Although this seems like a good idea, the reality is that people using a CASE tool know enough to double click on it, or whatever strategy the tool implements, to get more detail. The rake isn't adding any extra value.

9. Minimize the Number of Bubble Types

Koning, Dormann, and Van Vliet (2002) recommend that you have six or fewer bubbles on a diagram; any more risks overwhelming the user of the model.

10. Include White Space in Diagrams

White space is the empty areas between modeling elements on your diagrams. In the first version of Figure 2 the symbols are crowding each other, whereas in the second version, the symbols are spread out from one another, thus improving the

readability of the diagram. Observe that in the second version there is adequate space to add labels to the lines.

11. Organize Diagrams Left to Right, Top to Bottom

In Western cultures, people read left to right and top to bottom and therefore this is how they will read your diagrams. If there is a starting point for reading the diagram, such as the initial state of a UML state chart diagram or the beginning of the flow of logic on a UML sequence diagram, then place it toward the top left corner of your diagram and continue appropriately from there.

12. Avoid Many Close Lines

Several lines close together are hard to follow.

13. Provide a Notation Legend

If you're not sure that all of the users of a model understand the notation that you're using, provide them with a legend that overviews it. A good legend indicates the notational symbols used, the name of the symbol, and a description of its usage. Figure 4 provides an example for robustness diagrams (Jacobson, Christerson, Jonsson, and Overgaard 1992; Rosenberg and Scott 1999), a modification of UML communication diagrams (Chapter 8).

2.2 Simplicity Guidelines

14. Show Only What You Have to Show

Diagrams showing too many details are difficult to read because they are too information-dense. One of the practices of Agile Modeling (Chapter 17) is to *Depict Models Simply*: to include only critical information on your diagrams and to exclude anything extraneous. A simple model that shows the key features that you are trying to depict—perhaps a UML

GENERAL DIAGRAMMING GUIDELINES 9

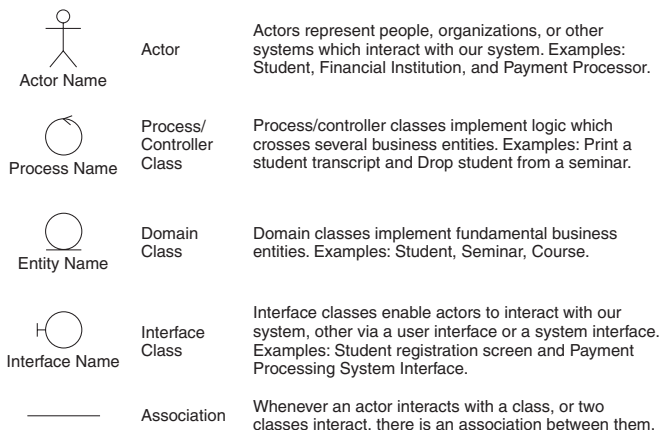


Figure 4. A legend for robustness diagrams.

class diagram depicting the primary responsibilities of classes and the relationships between them—often proves to be sufficient. Yes, you could model all of the scaffolding code that you will need to implement, but what value would that add? Very little.

15. Prefer Well-Known Notation over Esoteric Notation

Diagrams that include esoteric notation, instead of just the 20 percent “kernel notation” that does 80 percent of the job, can be difficult to read. Of course, what is well known in one organization may not be so well known in another, and so, you may want to consider supplying people with a brief summary of the notation that you’re using.

16. Reorganize Large Diagrams into Several Smaller Ones

It is often better to have several diagrams showing various degrees of detail than one complex diagram that shows

everything. A good rule of thumb is that a diagram shouldn't have more than nine symbols on it, based on the 7 ± 2 rule (Miller 1957), because there is a limit on the amount of information that someone can deal with at once. "Wallpaper" diagrams, particularly enterprise data models or enterprise object models, may look interesting but they're too information-dense to be effective. When you are reorganizing a large diagram into several smaller ones, you may choose to introduce a high-level UML package diagram (Chapter 6).

17. Prefer Single-Page Diagrams

To reduce complexity, a diagram should be printable on a single sheet of paper to help reduce its scope as well as to prevent wasted time cutting and taping several pages together. Be aware that you will reduce the usability of a diagram if you need to reduce the font too much or crowd the symbols and lines.

18. Focus on Content First, Appearance Second

There is always the danger of adding hours onto your CASE tool modeling efforts by rearranging the layout of your symbols and lines to improve the diagram's readability. The best approach is to focus on the content of a diagram at first and only try to get it looking good in a rough sort of way—it doesn't have to be perfect while you're working on it. Once you're satisfied that your diagram is accurate enough, and that you want to keep it, then invest the appropriate time to make it look good. An advantage of this approach is that you don't invest significant effort improving diagrams that you eventually discard.

19. Apply Consistent, Readable Fonts

Consistent, easy-to-read fonts improve the readability of your diagrams. Good ideas include fonts in the Courier, Arial,

and Times families. Bad ideas include small fonts (less than 10 point), large fonts (greater than 18 point), and italics.

2.3 Naming Guidelines

20. Set and Follow Effective Naming Conventions

This is one of the easiest things that you can do to ensure consistency within your models, and hence increase their readability.

21. Apply Common Domain Terminology in Names

Apply consistent and recognizable domain terminology, such as customer and order, whenever possible on your diagrams. This is particularly true for requirements and analysis-oriented diagrams with which your project stakeholders are likely to be working.

22. Apply Language Naming Conventions on Design Diagrams

Design diagrams should reflect implementation issues, including language naming conventions, such as *orderNumber* for an attribute and *sendMessage()* in Java. Requirements and analysis-oriented diagrams should not reflect language issues such as this.

23. Name Common Elements Consistently Across Diagrams

A single modeling element, such as an actor or a class, will appear on several of your diagrams. For example, the same class will appear on several UML class diagrams, several UML sequence diagrams, several UML communication diagrams, and several UML activity diagrams. This class should have the same name on each diagram; otherwise your readers will become confused.

2.4 General Guidelines

24. *Indicate Unknowns with a Question Mark*

While you are modeling, you may discover that you do not have complete information. This is particularly true when you are analyzing the domain. You should always try to track down a sufficient answer, but if you cannot do so immediately, then make a good guess and indicate your uncertainty. Figure 5 depicts a common way to do so with its use of question marks.¹ First, there is a UML note (see Section 3.1) attached to the association between *Professor* and *Seminar* questioning the multiplicity. Second, there is a question mark above the constraint on the *wait listed* association between *Student* and *Seminar*, likely an indication that the modeler isn't sure that it really is a first in, first out (FIFO) list.

25. *Consider Applying Color to Your Diagrams*

Coad, Lefebvre, and DeLuca (1999) provide excellent advice in their book *Java Modeling in Color with UML* for improving the understandability of diagrams by applying color to them, in addition to UML stereotypes. Perhaps color could indicate the implementation language of a class (e.g., blue for Java and red for C++) on a UML class diagram, the development priority of a use case (e.g., red for phase 1, orange for phase 2, and yellow for future phases) on a UML use case diagram, or the target platform (e.g., blue for an application server, green for a client machine, and pink for a database server) for a software element on a UML deployment diagram.

26. *Apply Color or Different Fonts Sparingly*

Evitts (2000) suggests the use of different fonts, line styles, colors, and shading to emphasize different aspects of your

¹ Question marks are not official UML notation.

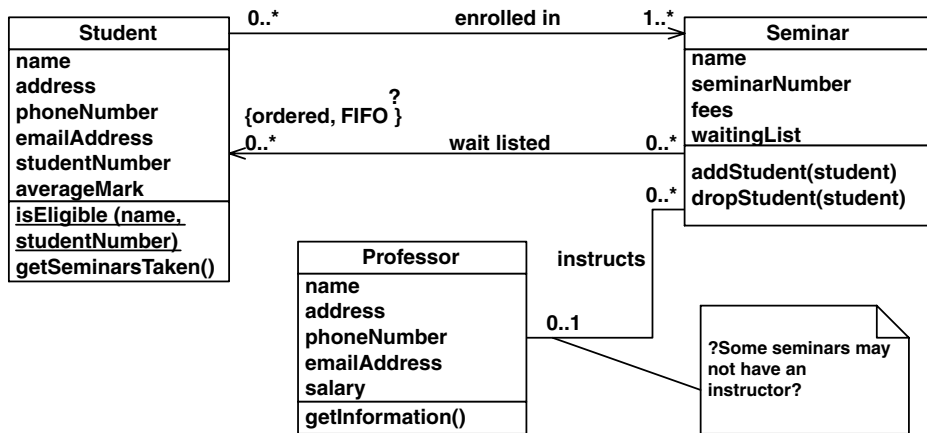


Figure 5. Indicating uncertainty on a diagram.

14 THE ELEMENTS OF UML 2.0 STYLE

diagrams. The secret is to do this sparingly; otherwise you run the risk of creating noisy/gaudy diagrams. Sometimes less is more. Koning, Dormann, and Van Vliet (2002) also suggest restraint when applying color, suggesting that you not use more than six colors in a single diagram, that you use vivid colors only for strong signaling, and that you prefer colors that are light.

3.

Guidelines for Common UML Modeling Elements

An important benefit of the UML is that it is consistent, and part of that consistency is the application of common modeling elements across different diagrams. This chapter describes guidelines for

- Notes
- Stereotypes
- Frames
- Interfaces

3.1 Guidelines for UML Notes

A UML note is a modeling construct for adding textual information—such as a comment, constraint definition, or method body—to UML diagrams. As you can see in Figure 6, notes are depicted as rectangles with the top right corners folded over.

27. Describe Diagrams with Notes

Ever look at a diagram and not know what it represents? A simple solution is to include a UML note on each diagram

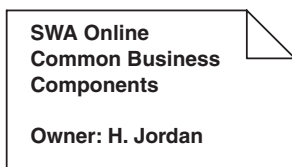


Figure 6. A summary note for a diagram.

that provides a simple and concise description. This is often referred to as a “legend.” In Figure 6 you can see that the name of the system, the purpose of the diagram, and its owner are indicated. It is common also to indicate when the diagram was last updated and the contact information for the owner.

28. Set a Convention for Placement of Diagram Legends

Placing diagram legends in the same place on all your diagrams increases their usability by making them easy to find. Common spots are one of the corners or the bottom center of a diagram.

29. Left-Justify Text in Notes

It is common practice to left-justify text in UML notes, as you can see in Figure 6.

30. Prefer Notes over OCL or ASL to Indicate Constraints

Source code, describing a constraint or processing logic, can be modeled on any UML diagram using a note. In UML, constraints are modeled either by a UML note using free-form text or with Object Constraint Language (OCL) (Warmer and Kleppe 2003). Future versions of UML are expected to support a common Action Semantic Language (ASL) for defining process logic—right now it is common to model tool-specific ASL, program source code (e.g., Java or C#), or structured English.

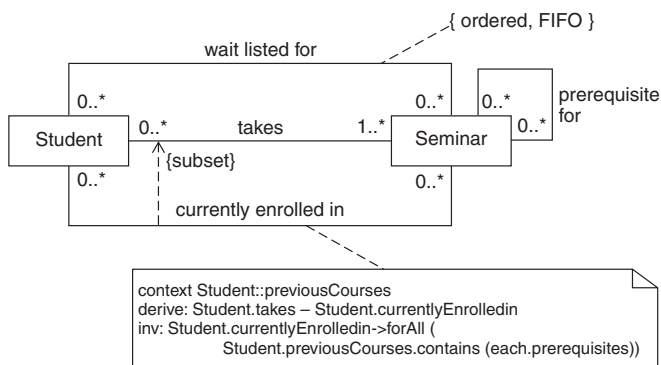


Figure 7. Indicating OCL on a UML diagram.

Figure 7 includes three examples of OCL—two as simple constraints on relationships and one as a note. The constraint `{ordered, FIFO}` indicates that the waiting list to get into a seminar is ordered in “first in, first out” sequence. The constraint `{subset}` indicates that the *currently enrolled in* relationship is a subset of the *takes* relationships. Both of these are reasonably straightforward to understand, although they would prove too abstract for some business stakeholders. The note contains an even more complex example of OCL, something that likely could be written and understood only by programmers. Sophisticated CASE tools will use the information contained in this note to generate working software, although if your CASE tool doesn’t do this then I question the value of writing this OCL.

It would have been easier to have just written the text “Students must have taken the prerequisite seminars for each one they are currently enrolled in.” When the audience for a diagram includes project stakeholders, you should write a free-form note, perhaps using natural language, for your constraint. Consider OCL or ASL for diagrams whose only audience is developers,

but recognize that this is only appropriate if everyone involved understands OCL or ASL.

31. Follow Common Coding Conventions for OCL and ASL

The book *The Elements of Java Style* (Vermeulen et al. 2000) provides Java coding guidance that you can modify for OCL and ASL.

3.2 Guidelines for UML Stereotypes

A stereotype denotes a variation on an existing modeling element with the same form but with a modified intent (Rumbaugh, Jacobson, and Booch 2004). Stereotypes are used to extend the UML in a consistent manner.

32. Name Stereotypes in `<<user interface>>` and `<<UI>>` Format

It is common UML convention to use lowercase for stereotypes that are spelled out fully, such as `<<include>>` instead of `<<Include>>`, and to use all uppercase for stereotypes that are abbreviations, such as `<<HTTP>>` instead of `<<Http>>`.

33. List Stereotypes Last

The most important information should always come first, followed by the next most important information, and so on. The stereotypes applicable to a model element should be listed to the right of or below the primary information. In Figure 8 the second version of the *Customer* class lists the stereotypes for its operations after the operation signature, not before it.

34. Don't Indicate Assumed Stereotypes

In Figure 8 I dropped the `<<business domain>>` stereotype because it is common practice to assume that a class is a business domain one unless marked otherwise.

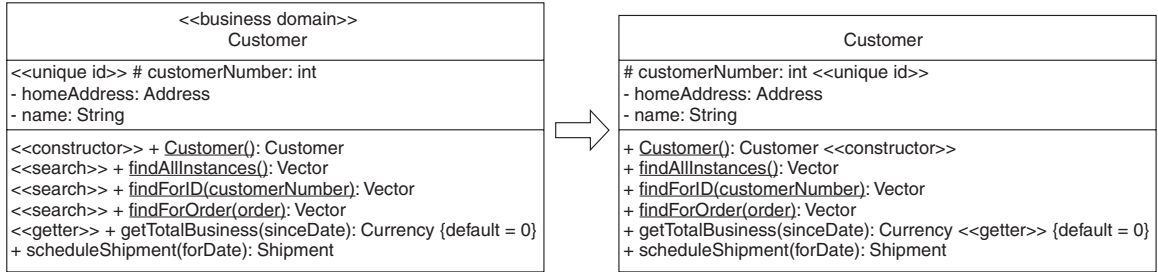


Figure 8. Indicating stereotypes.

35. *Prefer Naming Conventions over Stereotypes*

An effective alternative to applying a stereotype is to apply a naming convention. For example, instead of applying the stereotype `<<getter>>` on an operation, you could simply start all getters with the text *get*, as you can see in Figure 5 with the *getSeminarsTaken()* operation. This simplifies your diagrams and increases the consistency of your source code. Normally I would have not included `<<getter>>` in Figure 8 but I left it there for the discussion of Guideline #36. Notice how in Figure 8 the `<<search>>` stereotypes were removed because the operation names all start with *find*.

A drawback of this approach is that using naming conventions can hide the stereotyping information from your modeling tool. Ideally you should be able to define your naming conventions in your tool, enabling it to act intelligently, but that can be a lot to ask of the vendors.

36. *Tagged Values Follow Stereotypes*

In Figure 8 you see that the tagged value default follows the `<<getter>>` stereotype, because this level of detail is often the least important information shown on a diagram. Tagged values are sometimes referred to as named variables or named elements.

37. *Align Classifier Stereotypes with Names*

The stereotype for the *Customer* class in Figure 8 is centered because the name is centered. Had the name in the class been left justified then so should the stereotype have been. This helps to visually associate the two pieces of information.

38. *Introduce New Stereotypes Sparingly*

A common mistake made by UML novices is to apply stereotypes to everything, forcing them to introduce a plethora of new stereotypes. The end result is that their diagrams are

cluttered with stereotypes. Introduce a new stereotype to clarify an aspect of a model, but don't introduce one simply to "complete" your model.

39. Apply Stereotypes Consistently

You will find that you need to document your common stereotypes, above and beyond those defined by the UML standard, to ensure that they are applied consistently. For example, you need to decide whether you are going to use `<<user interface>>` or `<<UI>>` as your preferred stereotype. Both are good choices; choose one and move forward.

40. Apply Visual Stereotypes Sparingly

Figure 9 depicts a sequence diagram (Chapter 7), which includes the standard robustness diagram symbols (Jacobson, Christerson, Jonsson, and Overgaard 1992; Rosenberg and Scott 1999) that are commonly applied to UML communication diagrams (Chapter 8). The visual stereotypes—a stick figure for an actor, a circle with an arrowhead for a process class, and a circle on top of a line for business domain classes—are applied instead of text stereotypes such as `<<actor>>`. Apply visual stereotypes only when they are well known to the user(s) of your diagrams. It should be safe to assume that users can read a text stereotype, but it's not as safe to assume they understand the visual representations.

3.3 Guidelines for UML Frames

A frame in the UML encapsulates a collection of collaborating instances or refers to another representation of such. Frames are depicted as rectangles with notched descriptor boxes in the top left corners. Frames come in two flavors, diagram frames such as *Batch Transcript Printing* and combined fragment frames such as the *loop* frame, both in Figure 9. Diagram frames explicitly define the boundary of a diagram,

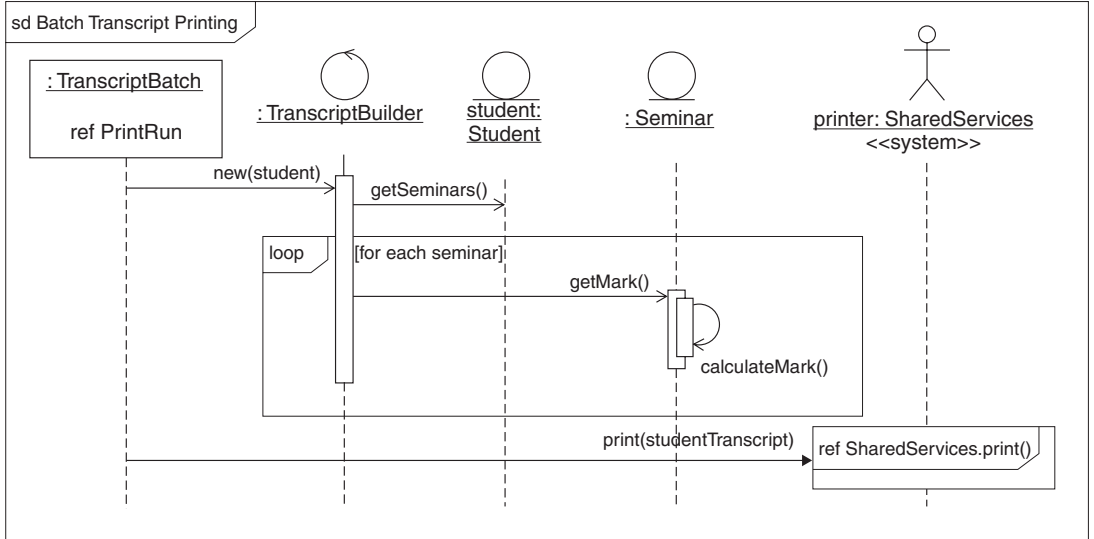


Figure 9. A frame encompassing a sequence diagram.

whereas combined fragment frames encompass portions of a diagram or provide references to other diagrams or method definitions.

41. Deemphasize Frame Borders

In Figure 9 you see that the frame border is lighter than the lines around it, reflecting the idea that the frame border is not as critical as the other information contained in the diagram.

42. Avoid Diagram Frames

The *Batch Transcript Printing* diagram frame of Figure 9 adds a significant amount of visual clutter. The only real value that it adds is that it indicates the name of the diagram, information that could have been captured in a note if it needed to be depicted at all. Diagram frames are in effect generic boundary boxes for diagrams, and that purpose is accomplished just as well by the edge of your computer screen, whiteboard, or paper.

Note that combined fragment frames, such as the loop in Figure 9, are very useful.

43. Apply Standard Labels to Descriptors

Table 1 summarizes common labels for diagram frames and Table 2 the common labels for combined fragments. Unfortunately many of these labels, which are part of the UML standard (Object Management Group 2004), are abbreviations, which decrease the readability of your diagrams.

44. Use Interaction Occurrences over Part Decompositions

There are two references to logic external to Figure 9: the *TranscriptBatch* object includes a reference to *PrintRun* and there is a combined fragment referencing the *SharedServices.print()* method. The style of the first reference is called a part

Table 1. Diagram Frame Labels

Label	Usage
Component	The frame depicts the internal design of a component.
Package	The frame depicts the internal organization of a package, often using a UML class diagram or a UML use case diagram
sd	Indicates that the frame contains an interaction diagram, usually a UML sequence diagram, although UML communication diagrams are also common options.
Use Case	The frame depicts the logic of a use case, often as a UML activity diagram or an interaction overview diagram.

decomposition and the second an interaction occurrence. As you can see, interaction occurrences are much more explicit than part decomposition and thus won't be overlooked as easily.

45. Fully Specify Operation Names in References

The reference to the *print* method in Figure 9 is presented in the full *classifier.operation()* format, making it explicit which operation (or in this case a service) is being invoked.

3.4 Guidelines for UML Interfaces

An interface is a collection of operation signatures and/or attribute definitions that ideally defines a cohesive set of behaviors. Interfaces are implemented, “realized” in UML parlance, by classes and components—to realize an interface, a class or component must implement the operations and attributes defined by the interface. Any given class or component may

Table 2. Combined Fragment Labels

Label	Usage
alt	Indicates several alternatives, only one of which will be taken, separated by dashed lines. Used to model <i>if</i> and <i>switch</i> statements.
assert	Indicates that the fragment models an assertion.
criticalRegion	Indicates that the fragment must be treated as atomic and cannot be interleaved with other event occurrences. Often used within a <i>par</i> frame (Douglass 2004).
loop	Models logic that will potentially be repeated several times.
opt	Models optional logic depending on the run-time evaluation of a guard.
par	Indicates several fragments of logic, separated by dashed lines, all of which will run in parallel. An example is presented in Figure 10.
ref	References another diagram or a method definition. This is often called an interaction use frame because this style of frame doesn't contain visual model elements, only text.

implement zero or more interfaces, and one or more classes or components can implement the same interface.

46. Depict One Interface per Port

Ports are connection points between a classifier and its environment, which are depicted on the side of frames as small rectangles. You may attach one or more interfaces to a port. In Figure 11 each port has exactly one interface, which is logically cohesive and thus does not reveal anything about the internal design of the *Seminar* component.

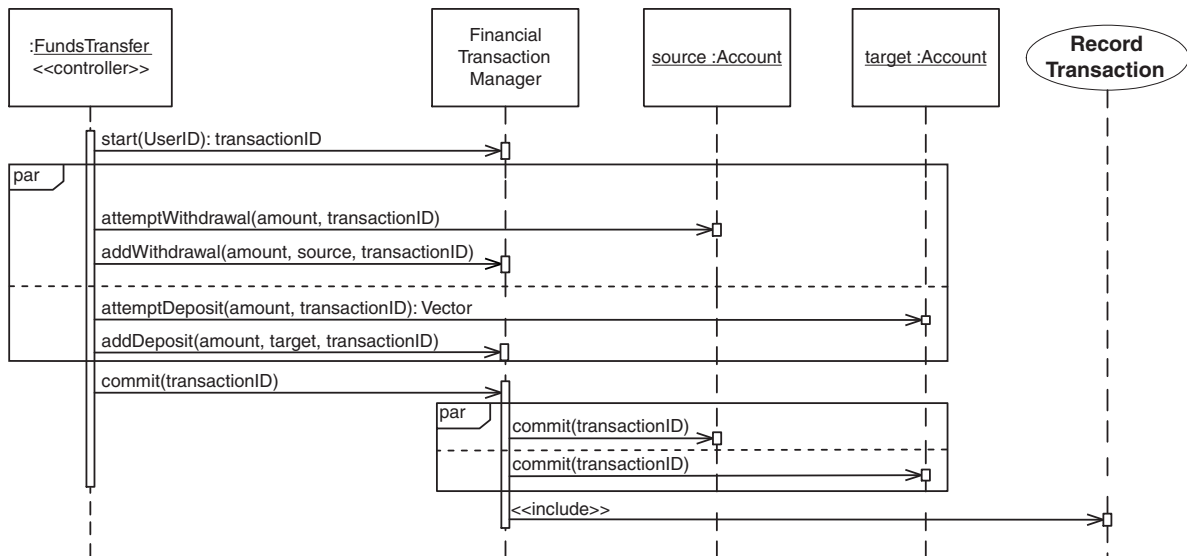


Figure 10. Transferring funds between accounts.

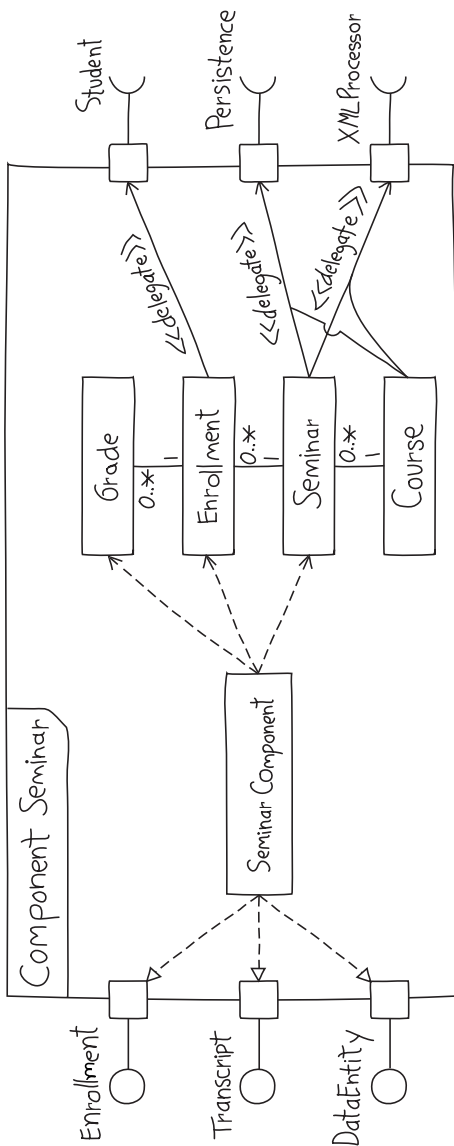


Figure 11. The internals of a (simple) component.

47. *Depict One Port per Realizing Class*

Another approach to organizing the interfaces for *SeminarComponent* of Figure 11 would be to have a single port offering the *Enrollment*, *Transcript*, and *DataEntity* interfaces. This would make for a more compact diagram, although it can contradict Guideline #46.

48. *Provided Interfaces on the Left*

Provided interfaces, those that a classifier implements, should be placed on the left of the box if possible. In Figure 11 the provided interfaces are depicted using “lollipop” notation. If you cannot fit a provided interface on the left side your next best option is the top of the box.

49. *Required Interfaces on the Right*

Required interfaces are those that a classifier must have provided for it to work. Required interfaces, such as *Student*, *Persistence*, and *XMLProcessor* in Figure 11 are depicted in UML 2 as “sockets.” The next best option is to place them on the bottom of the classifier. This rule, in combination with Guideline #48, makes it easier to create wiring-diagram style component diagrams (Chapter 11).

50. *Apply Realizes Relationships for Ports*

There are several ways to indicate that a classifier implements or requires the interfaces of a port. In Figure 11 *SeminarComponent* realizes the three ports on the left of the frame and the *Enrollment* class delegates to the *Student* port. It would have also been accurate to model the *Transcript* port as delegating to *SeminarComponent* or the *Student* port as realizing the required interface for the *Enrollment* class. We could also have used dependencies to model these relationships (dashed lines with arrowheads).

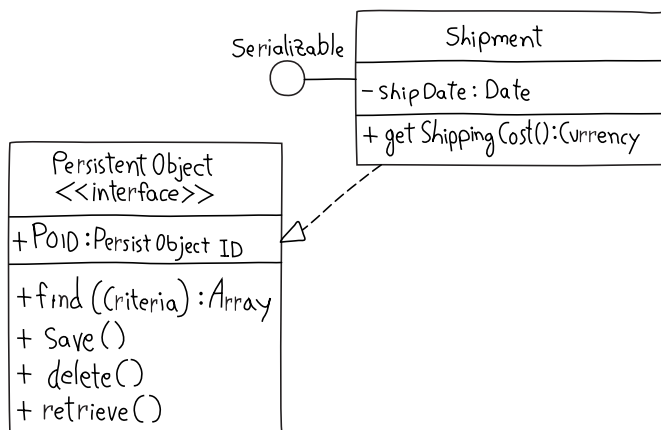


Figure 12. Interfaces on UML class diagrams.

51. Reflect Implementation Language Constraints in Interface Definitions

In Figure 12, you can see that a standard class box has been used to define the interface *PersistentObject* (note the use of the <<interface>> stereotype). This interface includes a public attribute named POID and several public operations. Unfortunately, it could not be implemented in Java because this language does not (yet) support instance attributes in the definition of interfaces. Therefore, you need to rework this interface definition if you wish to implement my model in Java.

52. Name Interfaces According to Language-Naming Conventions

Interfaces are named in the same manner as classes: they have fully described names in the format *InterfaceName*. In Java it is common to have interface names such as *Serializable* that end in *able* or *ible* or just descriptive nouns such as *EJBOject*.

In Microsoft environments, it is common practice to prefix interface names with a capital *I*, resulting in names such as *IComponent*.

53. Prefer “Lollipop” Notation to Indicate Realization of an Interface

As you can see in Figure 12, there are two ways to indicate that a class or component implements an interface: the lollipop notation used with the *Serializable* interface and the realization line (the dashed line with a closed arrowhead) used with the *PersistentObject* interface. The lollipop notation is preferred because it is visually compact; the class box and realization line approach tend to clutter your diagrams. Note that you’ll still need to model the specifics of the interface somewhere else.

54. Define Interfaces Separately from Your Classifiers

To reduce clutter, you can define interfaces separately from classifiers, either in another diagram specifically for interface definitions or simply on one edge of your class/component diagram.

55. Do Not Depict the Operations and Attributes of Interfaces in Your Classes

In Figure 12, you’ll notice that the *Shipment* class does not include the attributes or operations defined by the two interfaces that it realizes. That information would be redundant because it is already contained within the interface definitions. Note that a CASE tool should still include these elements in the definition of your classes, but that it shouldn’t display it.

56. One Label per Interface Connection

In Figure 13 you see that the *IPersistence* interface is indicated twice, once for the lollipop and once for the socket. When you

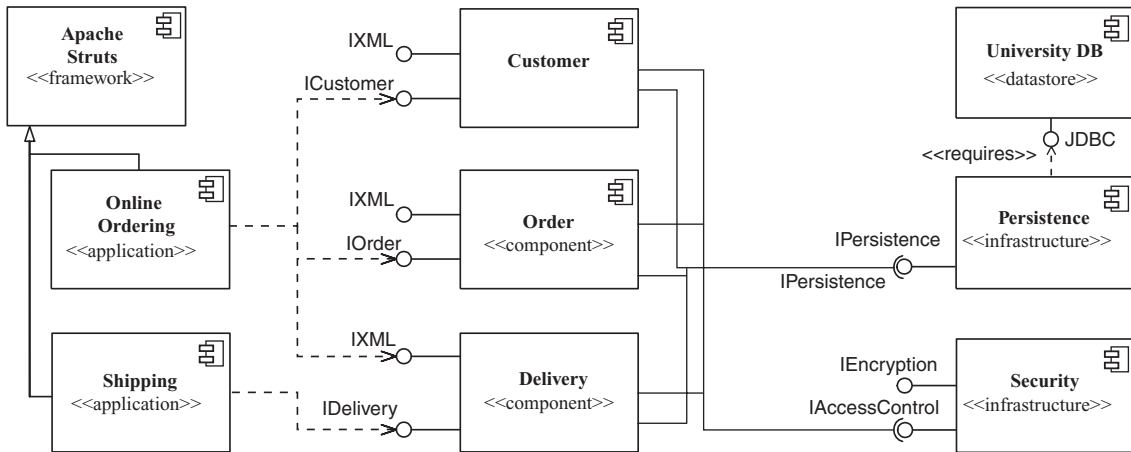


Figure 13. UML component diagram representing the logical architecture of a simple e-commerce system.

compare this with how the other interface names are modeled you see that listing it twice is unnecessary clutter.

57. Place Interface Labels Above the Interface

Whenever possible, interface labels should be placed above the interface lollipop/socket symbols.

5.

UML Class Diagrams

UML class diagrams show the classes of a system, their inter-relationships, and the operations and attributes of the classes. They are used to

- explore domain concepts in the form of a domain model,
- analyze requirements in the form of a conceptual/analysis model,
- depict the detailed design of object-oriented or object-based software.

A class model comprises one or more class diagrams and the supporting specifications that describe model elements, including classes, relationships between classes, and interfaces.

5.1 General Guidelines

Because UML class diagrams are used for a variety of purposes—from understanding your requirements to describing your detailed design—you will need to apply a different style in each circumstance. This section describes style guidelines pertaining to different types of class diagrams.

87. Identify Responsibilities on Domain Class Models

When creating a domain class diagram, often as part of your requirements modeling efforts, focus on identifying

Table 3. Visibility Options on UML Class Diagrams

Visibility	Symbol	Accessible to
Public	+	All objects within your system
Protected	#	Instances of the implementing class and its subclasses
Private	-	Instances of the implementing class
Package	~	Instances of classes within the same package

responsibilities for classes instead of on specific attributes or operations. For example, the *Invoice* class is responsible for providing its total, but whether it maintains this as an attribute or simply calculates it at request time is a design decision that you'll make later.

There is some disagreement about this guideline because it implies that you should be taking a responsibility-driven approach to development. Craig Larman (2002) suggests a data-driven approach, where you start domain models by identifying only data attributes, resulting in a model that is little different from a logical data model. If you need to create a logical data model, then do so, following AM's practice, *Apply the Right Artifact(s)* (Chapter 17). However, if you want to create a UML class diagram, then you should consider the whole picture and identify responsibilities.

88. Indicate Visibility Only on Design Models

The visibility of an operation or attribute defines the level of access that objects have to it, and the UML supports four types of visibility that are summarized in Table 3. Visibility is an important design issue. On detailed design models, you should always indicate the visibility of attributes and operations, an issue that typically is not pertinent to domain or conceptual models. Visibility on an analysis or domain model will always be public (+), and so there is little value in indicating this.

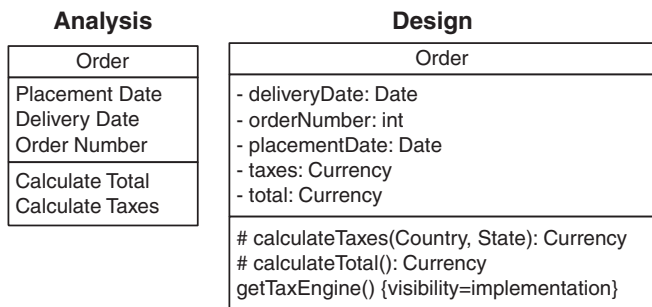


Figure 18. Analysis and design versions of a class.

89. Indicate Language-Dependent Visibility with Property Strings

If your implementation language includes non-UML-supported visibilities, such as C++'s implementation visibility, then a property string should be used, as you can see in Figure 18.

90. Indicate Types on Analysis Models Only When the Type Is an Actual Requirement

Sometimes the specific type of an attribute is a requirement. For example, your organization may have a standard definition for customer numbers that requires that they be nine-digit numbers. Perhaps existing systems, such as a legacy database or a predefined data feed, constrain some data elements to a specific type or size. If this is the case, you should indicate this information on your domain class model(s).

91. Be Consistent with Attribute Names and Types

It would not be consistent for an attribute named *customer-Number* to be a string, although it would make sense for it to be an integer. However, it would be consistent for the name *customerID* to be a string or an integer.

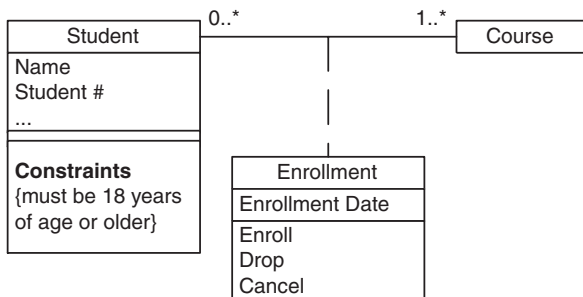


Figure 19. Modeling association classes.

92. Model Association Classes on Analysis Diagrams

Association classes, also called link classes, are used to model associations that have methods and attributes. Figure 19 shows that association classes are depicted as classes attached via dashed lines to associations—the association line, the class, and the dashed line are considered to be one symbol in the UML. Association classes typically are modeled during analysis and then refactored during design (either by hand or automatically by your CASE tool) because mainstream programming languages do not (yet) have native support for this concept.

93. Do Not Name Associations That Have Association Classes

The name of the association class should adequately describe it. Therefore, as you can see in Figure 19, the association does not need an additional adornment indicating its name.

94. Center the Dashed Line of an Association Class

The dashed line connecting the class to the association path should be clearly connected to the path and not to either class or to any adornments of the association, so that your

meaning is clear. As you can see in Figure 19, the easiest way to accomplish this is to center the dashed line on the association path.

5.2 Class Style Guidelines

A class is effectively a template from which objects are created (instantiated). Classes define attributes, information that is pertinent to their instances, and operations—functionality that the objects support. Classes also realize interfaces (more on this later). Note that you may need to soften some of these naming guidelines to reflect your implementation language or any purchased or adopted software.

95. *Use Common Terminology for Class Names*

Class names should be based on commonly accepted terminology to make them easier for others to understand. For business classes, this would include names based on domain terminology such as *Customer*, *OrderItem*, and *Shipment* and, for technical classes, names based on technical terminology such as *MessageQueue*, *ErrorLogger*, and *PersistenceBroker*.

96. *Prefer Complete Singular Nouns for Class Names*

Names such as *Customer* and *PersistenceBroker* are preferable to *Cust* and *PBroker*, respectively, because they are more descriptive and thus easier to understand. Furthermore, it is common practice to name classes as singular nouns such as *Customer* instead of *Customers*. Even if you have a class that does represent several objects, such as an iterator (Gamma, Helm, Johnson, and Vlissides 1995) over a collection of customer objects, a name such as *CustomerIterator* would be appropriate.

97. *Name Operations with Strong Verbs*

Operations implement the functionality of an object; therefore, they should be named in a manner that effectively

communicates that functionality. Table 4 lists operation names for analysis class diagrams as well as for design class diagrams—the assumption being that your implementation language follows Java naming conventions (Vermeulen et al. 2000)—indicating how the operation name has been improved in each case.

98. *Name Attributes with Domain-Based Nouns*

As with classes and operations, you should use full descriptions to name your attribute so that it is obvious what the attribute represents. Table 5 suggests a Java-based naming convention for analysis names that are in the *Attribute Name* format, although *attribute name* and *Attribute name* formats are also fine if applied consistently. Table 5 also suggests design names that take an *attributeName* format, although the *attribute_name* format is just as popular depending on your implementation language.

99. *Do Not Model Scaffolding Code*

Scaffolding code includes the attributes and operations required to implement basic functionality within your classes, such as the code required to implement relationships with other classes. Scaffolding code also includes getters and setters, also known as accessors and mutators, such as *getItem()* and *setItem()* in Figure 20, which get and set the value of attributes. You can simplify your class diagrams by assuming that scaffolding code will be created (many CASE tools can generate it for you automatically) and not model it. Figure 20 depicts the difference between the *OrderItem* class without scaffolding code and with it—including the constructor, the common static operation *findAllInstances()* that all business classes implement (in this system), and the attributes *item* and *order* and their corresponding getters and setters to maintain its relationships with the *Order* class and *Item* class, respectively.

Table 4. Example Names for Operations

Initial Name	Good Analysis Name	Good Design Name	Issue
Open Acc	Open Account	openAccount()	An abbreviation was replaced with the full word to make it clear what is meant.
Mailing Label Print	Print Mailing Label	printMailingLabel()	The verb was moved to the beginning of the name to make it active.
purchaseparkingpass()	Purchase Parking Pass	purchaseParkingPass()	Mixed case was applied to increase the readability of the design-level name.
Save the Object	Save	save()	The name was shortened because the term "TheObject" did not add any value.

Table 5. Example Names for Attributes

Initial Name	Good Analysis Name	Good Design Name	Issue
fName firstname	First Name First Name	firstName firstName	Do not use abbreviations in attribute names. Capitalizing the second word of the design name makes the attribute name easier to read.
personFirstName	First Name	firstName	This depends on the context of the attribute, but if this is an attribute of the “Person” class, then including “person” merely lengthens the name without providing any value.
nameLast	Last Name	lastName	The name “nameLast” was not consistent with “firstName” (and it sounded strange anyway).
HTTPConnection	HTTP Connection	httpConnection	The abbreviation for the design name should be in all lowercase.
firstNameString	First Name	firstName	Indicating the type of the attribute, in this case “string,” couples the attribute name to its type. If the type changes, perhaps because you decide to reimplement this attribute as an instance of the class “NameString,” then you will need to rename the attribute.
OrderItemCollection	Order Items	orderItems	The second version of the design name is shorter and easier to understand.

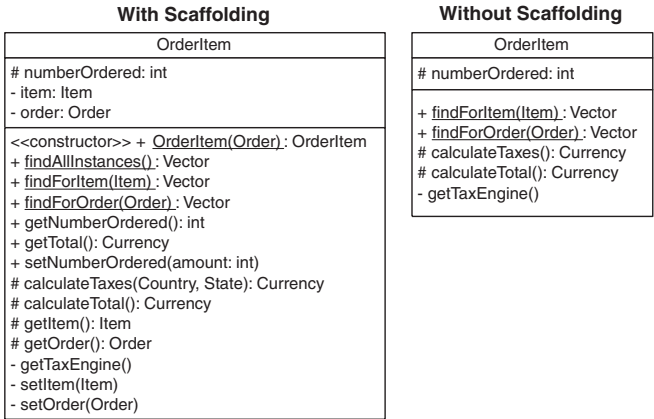


Figure 20. OrderItem class with and without scaffolding code.

100. Center Class Names

As you have seen in the figures in this chapter, it is common to center the names of classes.

101. Left-Justify Attribute Operation and Names

It is common practice, as you see in Figure 20 to left-justify both attribute and operation names within class boxes.

102. Do Not Model Keys

A key is a unique identifier of a data entity or table. Unless you are using a UML class diagram to model the logical or physical schema of a database (Ambler 2003), you should not model keys in your class. Keys are a data concept, not an object-oriented concept. A particularly common mistake of novice developers is to model foreign keys in classes, the data attributes needed to identify other rows of data within the database. A UML profile for physical data modeling is maintained at www.agiledata.org/essays/umlDataModelingProfile.html.

103. Never Show Classes with Just Two Compartments

It is allowable within the UML to have a class with one or more compartments. Although compartments may appear in any order, traditionally the topmost compartment indicates the name of the class and any information pertinent to the class as a whole (such as a stereotype); the second optional compartment typically lists the attributes; and the third optional compartment typically lists the operations. Other “nonstandard” compartments may be added to the class to provide information such as lists of exceptions thrown or notes pertaining to the class. Because naming conventions for attributes and operations are similar, and because people new to object development may confuse the two concepts, it isn’t advisable to have classes with just two compartments (one for the class name and one listing either attributes or operations). If necessary, include a blank compartment as a placeholder, as you can see with the *Student* class in Figure 19.

104. Label Uncommon Class Compartments

If you do intend to include a class compartment that isn’t one of the standard three—class name, attribute list, operations list—then include a descriptive label such as Exceptions, Examples, or Constraints centered at the top of the compartment, as you can see with the *Student* class in Figure 19.

105. Include an Ellipsis (...) at the End of an Incomplete List

You know that the list of attributes of the *Student* class of Figure 19 is incomplete because of the ellipsis at the end of the list. Without the ellipsis, there would be no indication that there is more to the class than what is currently shown (Evitts 2000).

106. List Static Operations/Attributes Before Instance Operations/Attributes

Static operations and attributes typically deal with early aspects of a class's life cycle, such as the creation of objects or finding existing instances of the classes. In other words, when you are working with a class you often start with statics. Therefore it makes sense to list them first in their appropriate compartments, as you can see in Figure 20 (statics are underlined).

107. List Operations/Attributes in Order of Decreasing Visibility

The greater the visibility of an operation or attribute, the greater the chance that someone else will be interested in it. For example, because public operations are accessible to a greater audience than protected operations, there is a likelihood that greater interest exists in public operations. Therefore, list your attributes and operations in order of decreasing visibility so that they appear in order of importance. As you can see in Figure 20, the operations and attributes of the *OrderItem* class are then listed alphabetically for each level of visibility.

108. For Parameters That Are Objects, List Only Their Types

As you can see in Figure 20, operation signatures can become quite long, extending the size of the class symbol. To save space, you can forgo listing the types of objects that are passed as parameters to operations. For example, Figure 20 lists *calculateTaxes(Country, State)* instead of *calculateTaxes(country: Country, state: State)*, thus saving space.

109. Develop Consistent Operation and Attribute Signatures

Operation names should be consistent with one another. For example, in Figure 20, all finder operations start with the

text *find*. Parameter names should also be consistent with one another. For example, parameter names such as *theFirstName*, *firstName*, and *firstNm* are not consistent with one another, nor are *firstName*, *aPhoneNumber*, and *theStudentNumber*. Pick one naming style for your parameters and stick to it. Similarly, be consistent also in the order of parameters. For example, the methods *doSomething(securityToken, startDate)* and *doSomethingElse(studentNumber, securityToken)* could be made more consistent by always passing *securityToken* as either the first or the last parameter.

110. Avoid Stereotypes Implied by Language Naming Conventions

The UML allows for stereotypes to be applied to operations. In Figure 20, I applied the stereotype `<<constructor>>` to the operation *OrderItem(Order)*, but that information is redundant because the name of the operation implies that it's a constructor, at least if the implementation language is Java or C++. Furthermore, you should avoid stereotypes such as `<<getter>>` and `<<setter>>` for similar reasons—the names *getAttributeName()* and *setAttributeName()* indicate the type of operations you're dealing with.

111. Indicate Exceptions in an Operation's Property String

Some languages, such as Java, allow operations to throw exceptions to indicate that an error condition has occurred. Exceptions can be indicated with UML property strings, an example of which is shown in Figure 21.

```
+ findAllInstances(): Vector
{exceptions=NetworkFailure, DatabaseError}
```

Figure 21. Indicating the exceptions thrown by an operation.

5.3 Relationship Guidelines

For ease of discussion the term relationships will include all UML concepts such as associations, aggregation, composition, dependencies, inheritance, and realizations. In other words, if it's a line on a UML class diagram, we'll consider it a relationship.

112. Model Relationships Horizontally

With the exception of inheritance, the common convention is to depict relationships horizontally. The more consistent you are in the manner in which you render your diagrams, the easier it will be to read them. In Figure 22, you can see that the dependencies are modeled horizontally, although the fulfilled via association is not. This sometimes happens.

113. Draw Qualifier Rectangles Smaller than Classes

Qualifiers, in this case *itemNumber* in Figure 22, are considered part of the association and not the classifier itself in the UML (even though *itemNumber* would likely be an attribute of *Item*). If possible the qualifier rectangle should be smaller than the class it is attached to (Object Management Group 2004).

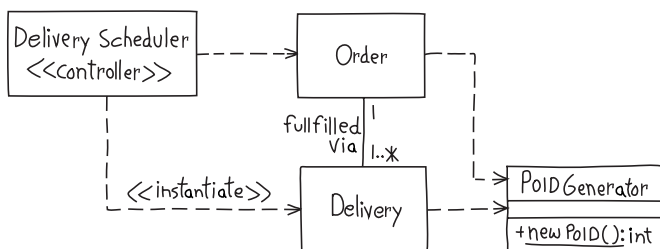


Figure 22. Shipping an order.

114. Model Collaboration Between Two Elements Only When They Have a Relationship

You need to have some sort of relationship between two model elements to enable them to collaborate. Furthermore, if two model elements do not collaborate with one another, then there is no need for a relationship between them.

115. Model a Dependency When the Relationship Is Transitory

Transitory relationships—relationships that are not persistent—occur when one or more of the items involved in a relationship is either itself transitory or a class. In Figure 22, you can see that there is a dependency between *DeliveryScheduler* and *Order*. *DeliveryScheduler* is a transitory class, one that does not need to persist in your database, and therefore, there is no need for any relationship between the scheduler and the order objects with which it interacts to persist. For the same reason, the relationship between *DeliveryScheduler* and *Delivery* is also a dependency, even though *DeliveryScheduler* creates *Delivery* objects.

In Figure 22, instances of *Delivery* interact with *OIDGenerator* to obtain a new integer value that acts as an object identifier (OID) to be used as a primary key value in a relational database. You know that *Delivery* objects are interacting with *OIDGenerator* and are not an instance of it because the operation is static. Therefore, there is no permanent relationship to be recorded, and so, a dependency is sufficient.

116. Tree-Route Similar Relationships to a Common Class

In Figure 22, you can see that both *Delivery* and *Order* have a dependency on *OIDGenerator*. Note how the two dependencies are drawn in combination in “tree configuration,” instead of as two separate lines, to reduce clutter in the diagram

(Evitts 2000). You can take this approach with any type of relationship. It is quite common with inheritance hierarchies (as you can see in Figure 25), as long as the relationship ends that you are combining are identical. For example, in Figure 23, you can see that *OrderItem* is involved in two separate relationships. Unfortunately, the *multiplicities* are different for each: one is 1..* and the other 0..*, so you can't combine the two into a tree structure. Had they been the same, you could have combined them, even though one relationship is aggregation and the other is association.

Note that there is a danger that you may be motivated to retain a relationship in order to preserve the tree arrangement, when you really should change it.

117. Always Indicate the Multiplicity

For each class involved in a relationship, there will always be a multiplicity. When the multiplicity is one and one only—for example, with aggregation and composition, it is often common for the part to be involved only with one whole—many modelers will not model the “1” beside the diamond. I believe that this is a mistake, and as you can see in Figure 23, I indicate the multiplicity in this case. If the multiplicity is “1,” then indicate it as such so that your readers know that you've considered the multiplicity. Table 6 summarizes the multiplicity indicators that you will see on UML class diagrams.

118. Avoid a Multiplicity of “”*

You should avoid the use of “*” to indicate multiplicity on a UML class diagram because your reader can never be sure if you really mean “0..*” or “1..*.” Although the UML specification (Object Management Group 2004) clearly states that “*” implies “0..*,” the reality is that you simply can't trust that everyone has read and memorized the several-hundred-page specification.

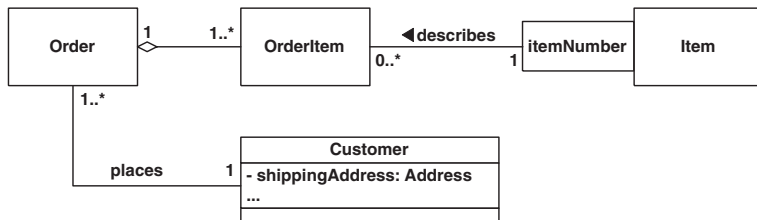


Figure 23. Modeling an order.

Table 6. UML Multiplicity Indicators

Indicator	Meaning
0..1	Zero or one
1	One only
0..*	Zero or more
1..*	One or more
<i>n</i>	Only <i>n</i> (where $n > 1$)
*	Many
0.. <i>n</i>	Zero to <i>n</i> (where $n > 1$)
1.. <i>n</i>	One to <i>n</i> (where $n > 1$)
<i>n</i> .. <i>m</i>	Where <i>n</i> and <i>m</i> both > 1
<i>n</i> ..*	<i>n</i> or more, where $n > 1$

119. Replace Relationship Lines with Attribute Types

In Figure 23, you can see that *Customer* has a *shippingAddress* attribute of type *Address*—part of the scaffolding code to maintain the association between customer objects and address objects. This simplifies the diagram because it visually replaces a class box and association, although it contradicts the *Do Not Model Scaffolding Code* guideline. You will need to judge which guideline to follow, the critical issue being which one will best improve your diagram given your situation.

A good rule of thumb is that if your audience is familiar with the class, then show it as a type. For example, if *Address* is a new concept within your domain then show it as a class; if it's been around for a while then showing it as a type should work well.

120. Do Not Model Implied Relationships

In Figure 23 there is an implied association between *Item* and *Order*—items appear on orders—but it was not modeled. A mistake? No; the association is implied through *OrderItem*. Orders are made up of order items, which in turn are described by items. If you model this implied association, not only do you clutter your diagram, but also you run the risk that somebody will develop the additional code to maintain

it. If you don't intend to maintain the actual relationship—for example, you aren't going to write the scaffolding code—then don't model it.

121. Do Not Model Every Dependency

Model a dependency between classes only if doing so adds to the communication value of your diagram. As always, you should strive to follow AM's (Chapter 17) practice, *Depict Models Simply*.

5.4 Association Guidelines

122. Center Names on Associations

It is common convention to center the name of an association above an association path, as you can see in Figure 23 with the *describes* association between *Order* and *Item*, or beside the path, as with the *fulfilled via* association between *Order* and *Delivery* in Figure 22.

123. Write Concise Association Names in Active Voice

The name of an association, which is optional although highly recommended, is typically one or two descriptive words. If you find that an association name is wordy, think about it from the other direction; for example, the *places* name of Figure 23 is concise when read from right to left but would be wordy if written from the left-to-right perspective (e.g., “is placed by”). Furthermore, *places* is written in active voice instead of passive voice, making it clearer to the reader

124. Indicate Directionality to Clarify an Association Name

When it isn't clear in which direction the name of an association should be read, you can indicate the direction with a filled triangle, as you can see in Figure 23 between *OrderItem* and

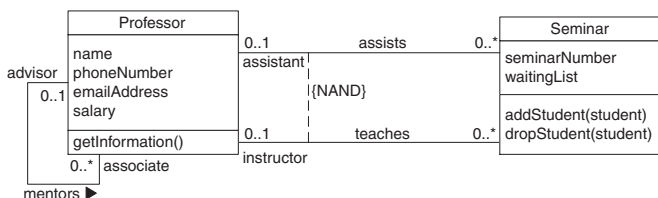


Figure 24. Professors and seminars.

Item. This marker indicates that the association should be read as “an item describes an order item” instead of “an order item describes an item.” It is also quite common to indicate the directionality on recursive associations, where the association starts and ends on the same class, such as *mentors* in Figure 24.

Better yet, when an association name isn’t clear, you should consider rewording it or maybe even renaming the classes.

125. Name Unidirectional Associations in the Same Direction

The reading direction of an association name should be the same as that of the unidirectional association. This is basically a consistency issue.

126. Word Association Names Left to Right

Because people in Western societies read from left to right, it is common practice to word association names so that they make sense when read from left to right. Had I followed this guideline with the *describes* association of Figure 23, I likely would not have needed to include the direction marker.

127. Indicate Role Names When Multiple Associations Between Two Classes Exist

Role names are optionally indicated on association ends to indicate how a class is involved in the association. Although the name of an association should make the roles of the two classes

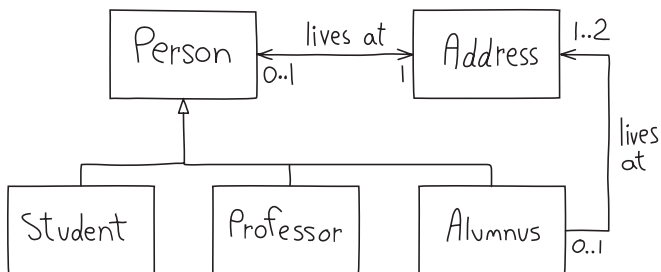


Figure 25. Modeling people in a university.

clear, it isn't always obvious when several associations exist between two classes. For example, in Figure 24, there are two associations between *Professor* and *Seminar*: *assists* and *teaches*. These two association names reflect common terminology at the university and cannot be changed. Therefore we opt to indicate the roles that professors play in each association to clarify them.

128. Indicate Role Names on Recursive Associations

Role names can be used to clarify recursive associations, ones that involve the same class on both ends, as you can see with the *mentors* association in Figure 24. The diagram clearly depicts the concept that an advisor mentors zero or more associate professors.

129. Make Associations Bidirectional Only When Collaboration Occurs in Both Directions

The *lives at* association between *Person* and *Address* in Figure 25 is unidirectional. A person object knows its address, but an address object does not know who lives at it. Within this domain, there is no requirement to traverse the association from *Address* to *Person*; therefore, the association does not need to be bidirectional (two-way). This reduces the code that

needs to be written and tested within the address class because the scaffolding to maintain the association to *Person* isn't required.

130. *Indicate Direction Only on Unidirectional Associations*

In Figure 25 the *lives at* association between *Person* and *Address* is unidirectional; it can be traversed from *Person* to *Address* but not in the other direction. The arrowhead on the line indicates the direction in which the association can be traversed. In Figure 24 all of the associations are bidirectional, yet the associations do not include direction indicators—when an association does not indicate direction it is assumed that it is bidirectional. I could have indicated an arrow on each end but didn't because this information would have been superfluous.

131. *Avoid Indicating Non-Navigability*

The X on the *lives at* association in Figure 25 indicates that you cannot navigate the association from *Address* to *Person*. However, because the association indicates that you can navigate from *Person* to *Address* (that's what the arrowhead means) the assumption would have been that you cannot navigate in the other direction. Therefore the X is superfluous.

132. *Redraw Inherited Associations Only When Something Changes*

An interesting aspect of Figure 25 is the association between *Person* and *Address*. First, this association was pushed up to *Person* because *Professor*, *Student*, and *Alumnus* all had a *lives at* association with *Address*. Because associations are implemented by the combination of attributes and operations, both of which are inherited, the implication is that associations are inherited. If the nature of the association doesn't change—for example, both students and professors live at only one address—then

we don't have any reason to redraw the association. However, because the association between *Alumnus* and *Address* is different, we have a requirement to track one or two addresses, and so we needed to redraw the association to reflect this.

133. Question Multiplicities Involving Minimums and Maximums

The problem with minimums and maximums is that they change over time. For example, today you may have a business rule that states that an alumnus has either one or two addresses that the university tracks, motivating you to model the multiplicity as 1..2, as depicted in Figure 25. However, if you build your system to reflect this rule, then when the rule changes you may find that you have significant rework to perform. In most object languages, it is easier to implement a 1..* multiplicity or, better yet, a 0..* multiplicity, because you don't have to check the size of the collection maintaining the association. Providing greater flexibility with less code seems good to me.

5.5 Inheritance Guidelines

Inheritance, also called generalization, models “is a” and “is like” relationships, enabling you to easily reuse existing data and code. When *A* inherits from *B*, we say that *A* is the subclass of *B* and that *B* is the superclass of *A*. Furthermore, we say that we have “pure inheritance” when *A* inherits all of the attributes and methods of *B*. The UML modeling notation for inheritance is a line with a closed arrowhead pointing from the subclass to the superclass.

134. Apply the Sentence Rule for Inheritance

One of the following sentences should make sense: “A subclass IS A superclass” or “A subclass IS KIND OF A superclass.” For example, it makes sense to say that a student is a person,

but it does not make sense to say that a student is an address or is like an address, and so, the class *Student* likely should not inherit from *Address*—association is likely a better option, as you can see in Figure 25. If it does not make sense to say that “the subclass is a superclass” or at least “the subclass is kind of a superclass,” then you are likely misapplying inheritance.

135. Place Subclasses Below Superclasses

It is common convention to place a subclass, such as *Student* in Figure 25, below its superclass—*Person* in this case. Evitts (2000) says it well: Inheritance goes up.

136. Beware of Data-Based Inheritance

If the only reason two classes inherit from each other is that they share common data attributes, then this indicates one of two things: either you have missed some common behavior (this is likely if the sentence rule applies) or you should have applied association instead.

137. A Subclass Should Inherit Everything

A subclass should inherit all of the attributes and operations of its superclass, and therefore all of its relationships as well—a concept called pure inheritance. The advantage of pure inheritance is that you only have to understand what a subclass inherits, and not what it does not inherit. Although this sounds trivial, in a deep class hierarchy it makes it a lot easier if you only need to understand what each class adds, and not what it takes away. Larman (2002) calls this the “100% rule.” Note that this contradicts the *Redraw Inherited Associations Only When Something Changes* guideline, and you’ll need to decide accordingly.

138. Differentiate Generalizations Between Associations

It is possible, albeit uncommon, for an association to inherit from another association. In the case you should use a different

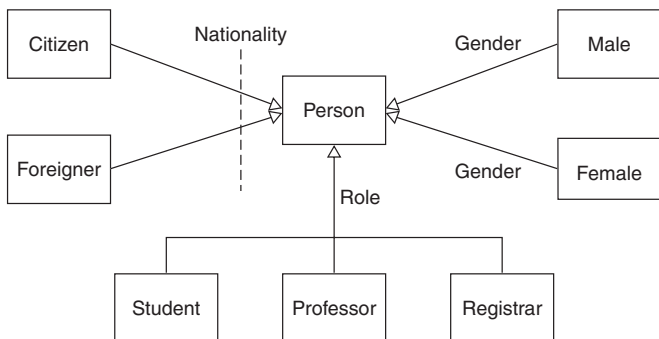


Figure 26. Indicating power types.

color or line weight for the generalization relationship to distinguish it (Object Management Group 2004).

139. Indicate Power Types on Shared Generalization

Power types, a meta modeling concept, can be indicated using a role name associated with a generalization relationship. Figure 26 shows three different ways to model power types. The preferred way was used to indicate that the *Student*, *Professor*, and *Registrar* classes are distinguished by the role that the person plays at the university, because it is concise. The same person may also be either a *Citizen* or a *Foreigner*, but not both. We know this because of the dashed line. The way that the *Gender* power type is indicated, on each of the generalization relationships, clutters the diagram.

5.6 Aggregation and Composition Guidelines

Sometimes an object is made up of other objects. For example, an airplane is made up of a fuselage, wings, engines, landing gear, flaps, and so on. A delivery shipment contains one or more packages. A team consists of two or more employees. These are all examples of the concept of aggregation, which

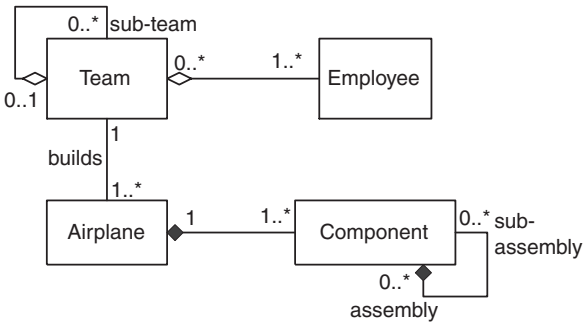


Figure 27. Examples of aggregation and composition.

represents “is part of” relationships. An engine is part of a plane, a package is part of a shipment, and an employee is part of a team. Aggregation is a specialization of association, specifying a whole–part relationship between two objects. Composition is a stronger form of aggregation, where the whole and the parts have coincident lifetimes, and it is very common for the whole to manage the life cycle of its parts. From a stylistic point of view, because aggregation and composition are both specializations of association, the guidelines for associations apply.

140. Apply the Sentence Rule for Aggregation

It should make sense to say “the part IS PART OF the whole.” For example, in Figure 27, it makes sense to say that a course is part of a program or that a component is part of an airplane. However, it does not make sense to say that a team is part of an airplane or an airplane is part of a team—but it does make sense to say that a team builds airplanes, an indication that association is applicable.

141. Be Interested in Both the Whole and the Part

For aggregation and composition, you should be interested in both the whole and the part separately—both the whole

and the part should exhibit behavior that is of value to your system. For example, you could model the fact that my watch has hands on it, but if this fact isn't pertinent to your system (perhaps you sell watches but not watch parts), then there is no value in modeling watch hands.

142. Place the Whole to the Left of the Part

It is a common convention to draw the whole, such as *Team* and *Airplane*, to the left of the part, such as *Employee* and *Component*, respectively.

143. Apply Composition to Aggregates of Physical Items

Composition is usually applicable whenever aggregation is *and* when both classes represent physical items. For example, in Figure 27 you can see that composition is used between *Airplane* and *Component*, whereas aggregation is used between *Team* and *Employee*—airplanes and components are both physical items, whereas teams are not.

144. Apply Composition When the Parts Share Their Persistence Life Cycle with the Whole

If the persistence life cycle of the parts is the same as that of the whole, if they're read in at the same time, if they're saved at the same time, if they're deleted at the same time, then composition is likely applicable.

145. Don't Worry About the Diamonds

When you are deciding whether to use aggregation or composition over association, Craig Larman (2002) says it best: "If in doubt, leave it out." The reality is that many modelers will agonize over when to use aggregation when the reality is that there is very little difference between association, aggregation, and composition at the coding level.

6.

UML Package Diagrams

A UML package diagram depicts two or more packages and the dependencies between them. A package is a UML construct that enables you to organize model elements, such as use cases or classes, into groups. Packages are depicted as file folders and can be applied on any UML diagram, although any diagram that depicts only packages (and their interdependencies) is considered a package diagram. UML package diagrams are in fact new to UML 2, although they were informally part of UML 1—what we called package diagrams in the past were in fact UML class diagrams or UML use-case diagrams consisting only of packages. Create a package diagram to

- depict a high-level overview of your requirements,
- depict a high-level overview of your design,
- logically modularize a complex diagram,
- organize source code,
- model a framework (Evitts 2000).

6.1 Class Package Diagram Guidelines

146. Create Class Package Diagrams to Logically Organize Your Design

Figure 28 depicts a UML package diagram that organizes a collection of related classes. In addition to the package guidelines

presented later in this chapter, apply the following heuristics to organize classes into packages:

- Classes of a framework belong in the same package.
- Classes in the same inheritance hierarchy typically belong in the same package.
- Classes related to one another via aggregation or composition often belong in the same package.
- Classes that collaborate with each other a lot often belong in the same package.

147. Create UML Component Diagrams to Physically Organize Your Design

If you have decided on a component-based approach to design, such as that promoted by Enterprise Java Beans (EJB) or Visual Basic, you should prefer a UML component diagram over a UML package diagram to depict your physical design. A version of Figure 28 as a UML component diagram is presented in Chapter 11 and, as you can see, that diagram is better suited to a physical design. Always remember to follow Agile Modeling's (Chapter 17) *Apply the Right Artifact(s)* practice.

148. Place Inheriting Packages Below Base Packages

Inheritance between packages is depicted in Figure 28 and, as you can see, the inheriting package is shown below the base package. This approach is consistent with other inheritance guidelines.

149. Vertically Layer Class Package Diagrams

Dependencies between packages indicate that the contents of the dependent package depend on, or have structural knowledge of, the contents of the other package. In Figure 28, the packages are placed on the diagram to reflect the logical layering of your architecture. The user interface interacts with

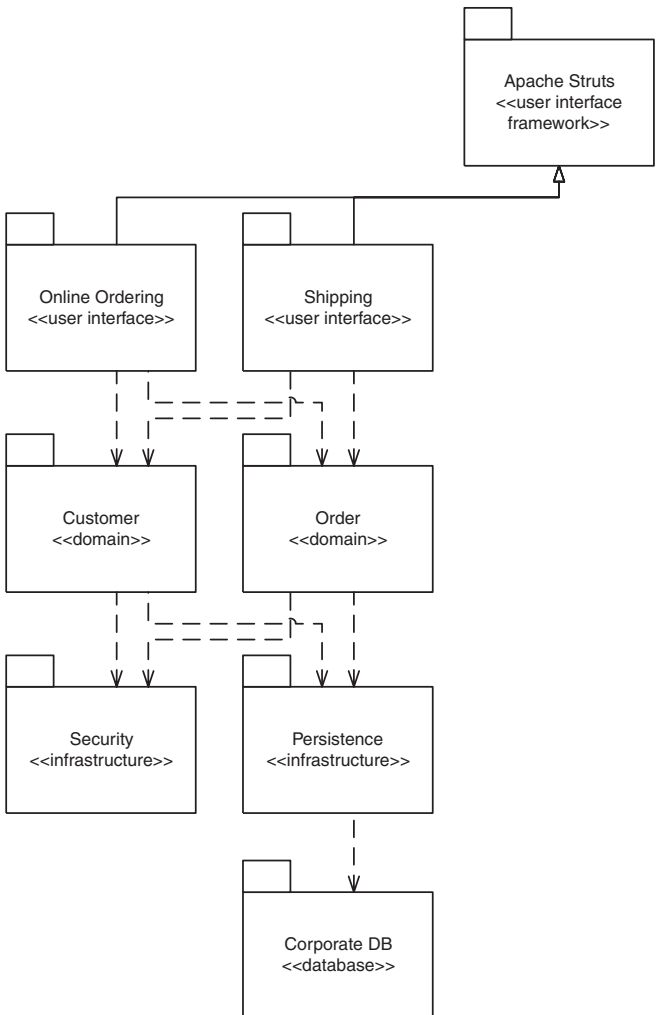


Figure 28. A package diagram organizing classes.

domain classes, which in turn use infrastructure classes, some of which access the database—which is traditionally depicted in a top-down manner.

6.2 Use-Case Package Diagram Guidelines

Use cases are a primary requirement artifact in many object-oriented development methodologies. This is particularly true of instantiations of the Unified Process (Rational Corporation 2002; Ambler, Nalbone, and Vizdos 2005). For larger projects, UML package diagrams are often created to organize these usage requirements.

150. Create Use-Case Package Diagrams to Organize Your Requirements

In addition to the package guidelines presented below, apply the following heuristics to organize UML use-case diagrams into package diagrams:

- Keep associated use cases together: included, extending, and inheriting use cases belong in the same package as the base/parent use case.
- Group use cases on the basis of the needs of the main actors. For example, in Figure 29, the *Enrollment* package contains use cases pertinent to enrolling students in seminars, a vital collection of services provided by the university.

151. Include Actors on Use-Case Package Diagrams

Including actors on UML package diagrams helps to put the packages in context, making diagrams easier to understand.

152. Arrange Use-Case Package Diagrams Horizontally

The primary audience of use-case package diagrams is project stakeholders; therefore, the organization of these diagrams

13.

UML Object Diagrams

UML object diagrams depict instances and their relationships at a point in time. You create a UML object diagram to

- explore “real-world” examples of objects and the relationships between them;
- explain complex relationships between classes to people who find class diagrams too abstract;
- become input into creating a UML class diagram.

287. Indicate Attribute Values to Clarify Instances

Figure 52 depicts a UML object diagram, taken from *The Object Primer 3rd Edition* (Ambler 2004), which explores the relationships between students and seminars. The student objects have been given understandable names, such as John Smith and Sally Jones, which immediately identify them. The seminar objects also have understandable names, such as CSC 100a, but also have attribute values such as Term = “Fall” to clarify which seminar is truly meant.

288. Prefer Object Names over Attribute Values

Whenever possible try to find a good, understandable name for an object without resorting to indicating attribute values. This makes your object diagrams concise.

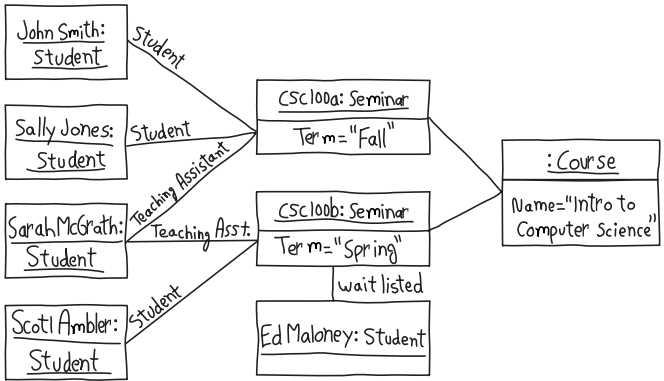


Figure 52. UML object diagram.

289. Indicate Roles to Distinguish Different Relationships

In Figure 52 you see that it is possible for students to be both teaching assistants and simply students of a seminar; for example, Sarah McGrath is a teaching assistant in CSC 100b, whereas Scott Ambler is a student of that seminar. Identifying roles in this manner can help you to identify different associations between the same classes.

14.

UML Composite Structure Diagrams

Composite structure diagrams are an addition to UML 2, although one style used to be referred to as an instance collaboration diagram. Composite structure diagrams are used to

- depict the internal structure of a classifier (such as a class, component, or use case), including the interaction points of the classifier to other parts of the system;
- explore how a collection of cooperating instances achieves a specific task or set of tasks;
- describe a design or architectural pattern or strategy.

There are two basic styles of UML composite structure diagram, the “collaboration style” depicted in Figure 53 from UML 1 and the new “detailed style” of Figure 54. Both diagrams model the same concept, the *Persist Object Via Framework* architectural strategy (Ambler 2003). Each box represents a lifeline and the dashed oval a collaboration. In the case of Figure 53 the focus is on the overall collaboration required to persist business objects, whereas in Figure 54 the focus is on the individual collaborations required to do so. The diagrams could in fact be combined—the detailed style of diagram could be encircled by a collaboration bubble.

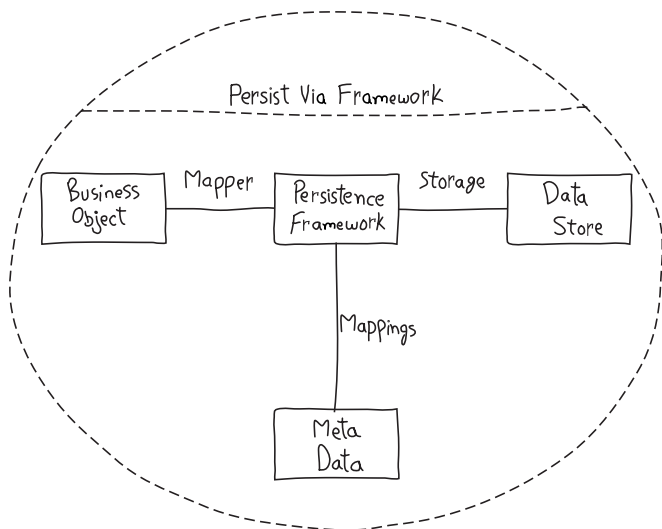


Figure 53. Collaboration-style composite structure diagram.

290. Focus on Object Roles in Collaboration-Style Composite Structure Diagrams

In collaboration-style composite structure diagrams, such as Figure 53, most of the value of the diagram comes in identifying the various roles that lifelines play in the collaboration.

291. Focus on Collaborations in Detailed-Style Composite Structure Diagrams

In detailed-style composite structure diagrams, such as the one depicted in Figure 54, the value comes from identifying the collaborations between lifelines.

292. Create Detailed-Style Composite Structure Diagrams over Collaboration Style

As you can see in the two examples, the detailed-style diagram contains more information than the collaboration-style diagram.

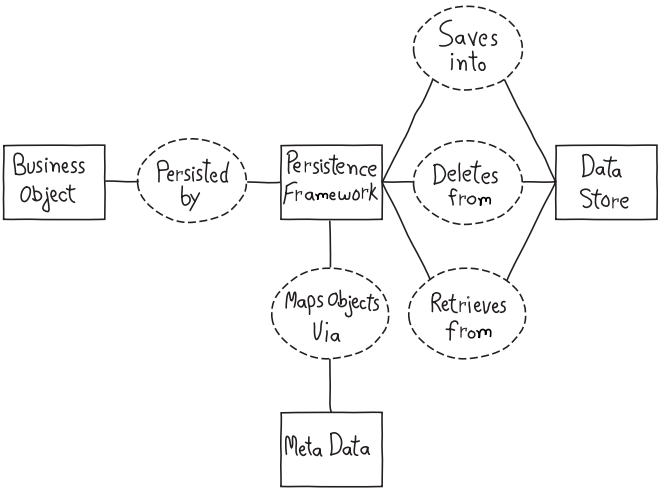


Figure 54. Detailed-style composite structure diagram.

293. Create UML Object Diagrams Instead of Composite Structure Diagrams

My experience is that few people use composite structure diagrams in practice. Frankly I don't see how they have any benefit that I couldn't get with UML object diagrams, which are much better known.